

Анализ применимости open-source решений к проекту

Заломный И.С.

Цель документа. Ответить на поставленные руководителем вопросы по open-source решениям защиты от сетевых атак: какие атаки они реально детектируют, на каком архитектурном уровне работают, какие признаки используют, и какие из их подходов мы можем перенести в наш проект.

Анализ ведётся применительно к текущему состоянию проекта (`bimka183/new_analizier`, `main` ветка): Go-бэкенд, парсер `pcap` на базе `gopacket`, разделение на потоки по 5-tuple и временные окна 10 секунд, набор детекторов в `src/detector` (`DDoSDetector`, `WormDetector`, `AdaptiveOverloadDetector`, `VirusDetector`), хранилище `SQLite`, фронтенд на `React`, оффлайн-режим работы.

1 Контекст и архитектурные ограничения проекта

Прежде чем оценивать применимость стороннего инструментария, надо явно зафиксировать рамки нашей системы — именно они определяют, что можно встроить, а что нет.

Режим работы. Система оффлайновая, *post-mortem*: пользователь загружает уже записанный `pcap`, анализатор разбирает его и отдаёт отчёт. Это исключает все *realtime*-сценарии вроде *inline IPS*, активного блокирования соединений, прямой интеграции с `iptables/nftables`.

Уровень доступа к трафику. Парсер строит две сущности — `FlowInfo` (поток по 5-tuple) и `TimeWindow` (агрегат за 10 секунд). На вход детекторам поступают только агрегированные структуры `FlowStats` и `WindowStats`: счётчики пакетов и байт, средний и стандартный размер пакета, `BPS`, `IAT`, длительность, счётчики `TCP`-флагов (`SYN/ACK/FIN/PSH/RST/URG`), количества уникальных `IP`/портов. Полезной нагрузки (`payload`) детекторы не видят — это означает, что любой подход, требующий поиска по содержимому пакетов, в текущей архитектуре нереализуем без расширения парсера.

Стек. Go в бэкенде, что ограничивает выбор библиотек: для большинства Python/C-инструментов прямого `binding`’а нет, либо он экспериментальный. Это не значит, что чужие инструменты нельзя использовать в принципе — их можно вызывать как внешний процесс над `pcap`-файлом и импортировать результаты. Но «встроить как библиотеку» получится только для того, что реализуемо средствами Go или имеет стабильные *cgo*-биндинги.

Хранилище. `SQLite`. Для индикаторов компрометации (`IoC`), результатов детекции, метаданных потоков — достаточно при оффлайн-сценарии работы с одним `pcap`.

Ниже все решения оцениваются именно в этих рамках.

2 Методология анализа

По каждому решению разбираем четыре пункта: что это и какие атаки оно реально детектирует; на каком архитектурном уровне работает (`packet` — отдельные пакеты и байты; `flow` —

агрегаты по 5-tuple; protocol/DPI — декодированные поля прикладного уровня; behavioral — поведенческий анализ нескольких потоков по хосту); какие признаки использует; применимо ли в нашем проекте и почему.

Выбраны два решения, представляющие принципиально разные архитектурные философии: Suricata — сигнатурный IDS/IPS с глубокой инспекцией пакетов; Zeek (Bro) — flow + протокол-семантика, программируемая логика. Эти два решения покрывают весь спектр подходов к NIDS/NIPS: от поиска по содержимому пакетов (Suricata) до анализа агрегированной статистики потоков (Zeek), что позволяет сделать обобщённые выводы о применимости разных подходов в нашем проекте.

3 Suricata

3.1 Как работает в общих чертах

Suricata — open-source NIDS/NIPS, разрабатывается некоммерческой OISF, лицензия GPL v2. Совместима с правилами Snort. Принцип работы — сигнатурный: каждое правило описывает признаки атаки, и движок проверяет каждый пакет (или реассемблированный поток) на соответствие правилам.

Пример типичного правила:

```
1 alert tcp any any -> $HOME_NET 445 (msg:"ETERNALBLUE Exploit Attempt";
2   content:"|FF|SMB|33 00 00 00 00 18|"; offset:4;
3   content:"|00 00 00 00 00 00 00 00 00 00 00 00 ff ff ff ff|"; distance:96;
4   sid:2024218;)
```

Правило ищет конкретную последовательность байт внутри payload SMB-пакета, идущего на порт 445. Найдено — алерт.

Suricata работает на всех уровнях одновременно: смотрит сырые байты, агрегирует на уровне потока через `flowbits` и `threshold`, декодирует прикладные протоколы (HTTP, DNS, TLS, SMB), может реассемблировать TCP-поток и извлекать из него файлы для матчинга по хешу. Большая часть детекторных возможностей основана на поиске по содержанию.

3.2 Почему Suricata не подходит нашему проекту

Главная причина — архитектурное несовпадение. Suricata смотрит внутрь пакетов, наш проект — на агрегированную статистику потоков.

Конкретно это означает следующее. Большая часть правил Suricata требует payload: сигнатуры эксплойтов, C2-каналов, веб-атак, аномалий протоколов — всё это байтовые паттерны внутри пакета. У нас payload не сохраняется в `FlowStats`, и без переработки парсера эти правила нельзя применить. Suricata нельзя встроить как библиотеку в Go: это отдельный демон на C, биндингов для Go в живом виде нет. Использовать её можно только как внешний процесс над `pcap`, но это превращает наш анализатор в обёртку над Suricata — мы теряем смысл собственного движка. Сильные стороны Suricata (DPI, JA3/JA4, file extraction) недоступны на нашем архитектурном уровне: они требуют доступа к содержимому, реассемблизации TCP-потока, парсеров прикладных протоколов.

Что частично можно использовать — это `threshold`-правила (типа «N SYN-пакетов от одного источника за T секунд») как источник проверенных порогов для наших собственных детекторов. Это `knowledge-base` в форме готовых правил, а не код. Полноценная же интеграция Suricata в наш проект не имеет смысла: либо мы строим собственный flow-анализатор (текущий путь), либо переходим на сигнатурный подход и принципиально меняем архитектуру.

Suricata — отличный инструмент в своей нише (сигнатурный DPI), но эта ниша не совпадает с нашей. Использовать её ценность мы можем только как источник готовых порогов для наших heuristic-детекторов.

4 Zeek (бывший Bro)

4.1 Что это и какие атаки детектирует

Zeek — open-source NSM-платформа, лицензия BSD, родом из академической среды (LBNL). В отличие от Suricata, Zeek не сигнатурный — его сила в том, что он раскладывает трафик на структурированные логи (`conn.log`, `dns.log`, `http.log`, `ssl.log`, `smb.log` и т.д.) и предоставляет язык скриптов для написания произвольной детекторной логики поверх этих логов.

Конкретно через встроенные политики Zeek детектирует следующее. Сканирование портов и адресов — скрипт `scan.zeek` реализует алгоритм TRW (Threshold Random Walk), статистически обоснованный способ отличить сканер от легитимного клиента уже после 4–5 попыток подключения. Это академическая работа Jung, Paxson et al. 2004, до сих пор state-of-the-art по соотношению скорость/точность. SSH brute force — `protocols/ssh/detect-bruteforcing.zeek`, через подсчёт неудачных попыток аутентификации. DNS-аномалии — слишком длинные имена, аномальные TTL, DNS-туннели через объёмные или частые запросы. TLS-аномалии — неожиданные SNI, самоподписанные сертификаты, expired certs, weak ciphers. SMB lateral movement — через `smb.log` и кастомные политики. DDoS — через подсчёт connection-state'ов в `conn.log` (большое количество S0 или REJ за окно). C2-каналы — через периодичность подключений (есть пакеты-надстройки, например от Corelight).

4.2 Уровень работы

Zeek работает преимущественно на flow-уровне: его основной выход — `conn.log`, набор flow-records со state-машиной (S0, S1, SF, REJ, RSTO, OTH и так далее). На уровне packet работает минимально, только для низкоуровневых событий. На уровне protocol/DPI работает сильно: встроенные парсеры HTTP, DNS, TLS, SMB, FTP, SSH выдают семантические события. На уровне behavioral — через скрипты, которые делают суммы и распределения по хостам и времени.

Ключевая особенность Zeek — connection state vocabulary. Каждому потоку присваивается одна из примерно четырнадцати буквенных кодов, обобщающих TCP-handshake и завершение. Основные коды: S0 — попытка SYN без ответа (типично для сканера); REJ — ответ RST, порт закрыт; SF — нормальное установление и закрытие; RSTO — originator прервал соединение; RSTR — responder прервал соединение; OTH — нет SYN, midstream-трафик.

Это уже само по себе мощный диагностический сигнал, на который можно строить детекторы: вместо «у нас 100 SYN и 0 ACK в окне» получаем «у нас 100 потоков в state S0 от одного источника» — что куда понятнее и точнее.

4.3 Какие признаки использует

Zeek опирается на полные flow stats: bytes, packets, duration в обе стороны (orig/resp). Использует TCP history string — последовательность флагов в порядке появления (ShADadFf и т.п.). Использует connection state code (S0/SF/REJ/...). Использует декодированные протокольные поля. В скриптах считаются производные метрики: энтропия dst-портов на хост, частоты подключений, размеры запросов и ответов.

4.4 Применимость в нашем проекте

Zeek с архитектурной точки зрения гораздо ближе к нашему анализатору, чем Suricata. Его философия — работать со структурированными flow-записями плюс пользовательской логикой над ними — это, по сути, та же самая модель, что у нас.

Совместимы и переносимы напрямую следующие компоненты: connection state codes (выводимы из наших счётчиков TCP-флагов); алгоритм TRW для port scan (работает на нашем уровне данных); Community ID (стандартный хеш над 5-tuple, не требует ничего кроме того, что у нас уже есть); threshold-логика для DDoS в стиле «много потоков в S0 за окно» (прямая адаптация); экспорт в формат `conn.log` (открывает интероп со всей экосистемой).

Расширения парсера до DPI требуют семантические логи (`dns.log`, `http.log`, `ssl.log`) и детекторы на их основе.

Раздел 5 ниже — конкретный план, что и как мы добавляем в наш проект.

5 План адаптации Zeek-подходов в наш проект

Здесь — конкретные пороговые значения, алгоритмы и места в коде, куда что встраивается. Это рабочая инструкция для разработчика.

5.1 Connection state code в FlowStats

Что добавляем — поле `ConnState string` в структуре `FlowStats`. Заполняется на этапе `AnalyzeFlow`.

Как считать — маппинг из счётчиков TCP-флагов в state-код. Сейчас наш бифлов складывает флаги обоих направлений в общие счётчики (`CntSYN`, `CntACK`, ...). Для аккуратного маппинга в Zeek-стиле нужно в `AnalyzeFlow` различать направление каждого пакета (`orig→resp` или `resp→orig` — определяется сравнением с первым пакетом потока).

Предлагается добавить в `FlowStats` парные счётчики:

```
1 type FlowStats struct {
2     // ... существующие поля ...
3
4     // Направление originator (тот, кто инициировал поток)
5     CntSYN_orig, CntACK_orig, CntFIN_orig, CntRST_orig int
6
7     // Направление responder
8     CntSYN_resp, CntACK_resp, CntFIN_resp, CntRST_resp int
9
10    ConnState string // "S0", "SF", "REJ", "RST0", и т.д.
11 }
```

Логика маппинга, упрощённая до основных состояний, которых достаточно для детекторов:

```
1 func deriveConnState(s *FlowStats) string {
2     // SYN от инициатора, ничего в ответ
3     if s.CntSYN_orig >= 1 && s.CntSYN_resp == 0 &&
4         s.CntACK_resp == 0 && s.CntRST_resp == 0 {
5         return "S0"
6     }
7     // SYN от инициатора, RST в ответ - порт закрыт
8     if s.CntSYN_orig >= 1 && s.CntRST_resp >= 1 {
9         return "REJ"
10    }
11    // Полный three-way handshake + нормальное закрытие
12    if s.CntSYN_orig >= 1 && s.CntSYN_resp >= 1 &&
13        s.CntFIN_orig >= 1 && s.CntFIN_resp >= 1 {
```

```

14         return "SF"
15     }
16     // Соединение установлено, originator прервал через RST
17     if s.CntSYN_orig >= 1 && s.CntACK_resp >= 1 && s.CntRST_orig >= 1 {
18         return "RSTO"
19     }
20     // Соединение установлено, responder прервал через RST
21     if s.CntSYN_orig >= 1 && s.CntACK_resp >= 1 && s.CntRST_resp >= 1 {
22         return "RSTR"
23     }
24     // Hem SYN - midstream
25     if s.CntSYN_orig == 0 && s.CntSYN_resp == 0 {
26         return "OTH"
27     }
28     // По умолчанию
29     return "OTH"
30 }

```

Куда класть — расширить `pkt.AnalyzeFlow` в `src/packet/`. После подсчёта направленных счётчиков вызвать `deriveConnState` и записать в `Stats.ConnState`.

Ценность — все остальные детекторы становятся выразительнее: вместо ручных порогов по сырым счётчикам флагов используем семантически осмысленные коды.

5.2 Port Scan Detector через TRW (Threshold Random Walk)

Что добавляем — новый файл `src/detector/portscan.go`, реализующий алгоритм Jung et al. 2004.

Принцип работы. Для каждого `SrcIP` отслеживаем серию его «первых контактов» с разными `dst` (то есть первое соединение с парой (`DstIP`, `DstPort`)). Каждый контакт классифицируется как успех или неудача. Успех — это `ConnState == "SF"` или `"S1"` (соединение состоялось). Неудача — это `ConnState == "S0"` или `"REJ"` (попытка отбита или нет ответа).

Поддерживаем для каждого `SrcIP` логарифмический счётчик отношения правдоподобия Λ . На каждом первом контакте обновляем Λ : прибавляем $\ln(\theta_1/\theta_0)$ если неудача, прибавляем $\ln((1 - \theta_1)/(1 - \theta_0))$ если успех. Если Λ превысило верхний порог η_1 — это сканер. Если опустилось ниже нижнего порога η_0 — точно не сканер, забываем.

Параметры из оригинальной работы. Допустимая доля ложных срабатываний $\alpha = 0.01$. Допустимая доля пропусков $\beta = 0.01$. Доля неудачных коннектов у нормального клиента $\theta_0 = 0.2$. Доля неудачных коннектов у сканера $\theta_1 = 0.8$.

Производные пороги в логарифмическом пространстве. Верхний порог $\ln \eta_1 = \ln((1 - \beta)/\alpha) \approx +4.6$. Нижний порог $\ln \eta_0 = \ln(\beta/(1 - \alpha)) \approx -4.6$. Шаг при неудаче $\ln(\theta_1/\theta_0) = \ln(0.8/0.2) \approx +1.386$. Шаг при успехе $\ln((1 - \theta_1)/(1 - \theta_0)) = \ln(0.2/0.8) \approx -1.386$.

Что это даёт на практике. Сканер с четырьмя неудачами подряд: $\Lambda = 4 \times 1.386 = 5.54 > 4.6$ — сработка после четырёх неудачных попыток. Это объясняет, почему TRW в десятки раз быстрее простых пороговых правил, при том же или меньшем числе ложных срабатываний.

Структура реализации:

```

1 package detector
2
3 import (
4     pkt "analizier/src/packet"
5 )
6
7 const (
8     trwLogEta1      = 4.6      // верхний порог
9     trwLogEta0      = -4.6     // нижний порог
10    trwStepFail      = 1.386    // ln(theta_1/theta_0) = ln(0.8/0.2)
11    trwStepSuccess   = -1.386   // ln((1-theta_1)/(1-theta_0)) = ln(0.2/0.8)

```

```

12 )
13
14 type PortScanDetector struct {
15     // Состояние по каждому SrcIP
16     state map[string]float64
17     // Уже сделанные первые контакты, чтобы не считать повторы
18     contacted map[string]map[string]bool // SrcIP -> set("DstIP:DstPort")
19     // Найденные сканеры
20     scanners map[string]bool
21 }
22
23 func NewPortScanDetector() *PortScanDetector {
24     return &PortScanDetector{
25         state:      make(map[string]float64),
26         contacted:   make(map[string]map[string]bool),
27         scanners:    make(map[string]bool),
28     }
29 }
30
31 // Analyze принимает все flows, отсортированные по времени первого пакета.
32 func (d *PortScanDetector) Analyze(flows []*pkt.FlowInfo) []string {
33     for _, flow := range flows {
34         src := flow.Stats.SrcIP
35         target := flow.Stats.DstIP + ":" + flow.Stats.DstPort
36
37         // Только первый контакт с конкретной парой (DstIP, DstPort)
38         if d.contacted[src] == nil {
39             d.contacted[src] = make(map[string]bool)
40         }
41         if d.contacted[src][target] {
42             continue
43         }
44         d.contacted[src][target] = true
45
46         // Уже признанный сканер не пересматривается
47         if d.scanners[src] {
48             continue
49         }
50
51         // Классификация исхода по ConnState
52         var step float64
53         switch flow.Stats.ConnState {
54             case "SF", "S1":
55                 step = trwStepSuccess
56             case "SO", "REJ", "RSTO", "SH":
57                 step = trwStepFail
58             default:
59                 continue // неинформативные states пропускаем
60         }
61
62         d.state[src] += step
63
64         // Проверка границ
65         if d.state[src] >= trwLogEta1 {
66             d.scanners[src] = true
67         } else if d.state[src] <= trwLogEta0 {
68             // Доказано легитимный - сбросим состояние
69             delete(d.state, src)
70         }
71     }
72
73     result := make([]string, 0, len(d.scanners))
74     for ip := range d.scanners {

```

```

75     result = append(result, ip)
76 }
77 return result
78 }

```

Куда добавить в `main.go` — после цикла `AnalyzeFlow` и перед существующими детекторами. Передаём детектору отсортированный по времени список `flows` и получаем список IP-сканеров.

Что отчёт должен показать — список `srcIP`, помеченных как сканер, с количеством первых контактов до момента детекции и значением финального `Λ`.

5.3 Community ID — стандартный hash 5-tuple

Что добавляем — поле `CommunityID string` в `FlowInfo`. Заполняется в `DivideByFlow` или сразу при создании потока.

Зачем. Это стандартный хеш, придуманный Corelight (за командой Zeek), который вычисляется детерминированно над нормализованным 5-tuple. Это позволяет нашему отчёту использовать тот же идентификатор потока, что Zeek, Suricata и SIEM-системы — мгновенно открывает интерпретацию.

Алгоритм по спецификации `corelight/community-id-spec`. Шаг 1: нормализовать пару `(srcIP, srcPort) ↔ (dstIP, dstPort)` так, чтобы на первом месте была меньшая (по байтовому сравнению). Шаг 2: сериализовать в фиксированный буфер — `seed` (2 байта, `0x0000`), затем `srcIP`, затем `dstIP`, затем `proto` (1 байт), затем `pad` (1 байт, `0x00`), затем `srcPort` (2 байта), затем `dstPort` (2 байта). Шаг 3: посчитать SHA-1 от буфера. Шаг 4: закодировать в base64. Шаг 5: дописать префикс `1:`.

Реализация на Go — около 40 строк, ничего необычного.

```

1 import (
2     "crypto/sha1"
3     "encoding/base64"
4     "encoding/binary"
5     "net"
6 )
7
8 const communityIDSeed uint16 = 0
9
10 func CommunityID(srcIP, dstIP net.IP, srcPort, dstPort uint16, proto uint8)
11     string {
12     // Нормализация направления: меньшая (IP, port) пара впереди
13     if compareTuple(srcIP, srcPort, dstIP, dstPort) > 0 {
14         srcIP, dstIP = dstIP, srcIP
15         srcPort, dstPort = dstPort, srcPort
16     }
17
18     var buf []byte
19     buf = binary.BigEndian.AppendUint16(buf, communityIDSeed)
20     buf = append(buf, srcIP.To16()...)
21     buf = append(buf, dstIP.To16()...)
22     buf = append(buf, proto)
23     buf = append(buf, 0x00) // padding
24     buf = binary.BigEndian.AppendUint16(buf, srcPort)
25     buf = binary.BigEndian.AppendUint16(buf, dstPort)
26
27     sum := sha1.Sum(buf)
28     return "1:" + base64.StdEncoding.EncodeToString(sum[:])
29 }

```

Куда положить — новый файл `src/packet/community_id.go`. Вызов в `DivideByFlow` при первом создании `FlowInfo`.

5.4 DDoS Detector в Zeek-стиле

Что улучшаем. Текущий DDoSDetector смотрит на WindowStats и порог BPS/PPS. Это работает для объёмных DDoS, но плохо для SYN flood и amplification. Дополним его проверкой по connection state.

Принцип Zeek. За временное окно считается количество уникальных потоков с state S0 или REJ, направленных в один dstIP. Если их больше порога — это либо DDoS, либо port scan от множества источников (DDoS-like).

Пороговые значения, взятые как стартовые от Zeek's scan.zeek SumStats-фреймворка и адаптированные под 10-секундное окно. Окно агрегации — 10 секунд (совпадает с нашим windows). Минимум потоков S0 или REJ к одному dstIP — 25 (стартовый порог Zeek для address scan). Доля S0+REJ среди всех потоков к dstIP — больше 0.7 (отличает DDoS от нормального трафика с потерями). Минимум потоков SYN от одного srcIP — 100 (PPS-сигнатура для SYN flood).

Что добавить в DDoSDetector:

```
1 type DDoSDetector struct {
2     // существующие поля порогов BPS/PPS
3
4     MinFailedFlowsPerDst int // 25
5     FailedRatio          float64 // 0.7
6 }
7
8 type DDoSAlert struct {
9     Window      pkt.TimeWindow
10    TargetIP     string
11    FlowsTotal   int
12    FlowsS0      int
13    FlowsREJ     int
14 }
15
16 func (d *DDoSDetector) AnalyzeFailedFlows(
17     windows []pkt.TimeWindow,
18     flows map[string]*pkt.FlowInfo,
19 ) []DDoSAlert {
20     var alerts []DDoSAlert
21
22     for _, win := range windows {
23         // Группируем потоки этого окна по dstIP
24         perDst := make(map[string]struct{ total, s0, rej int })
25
26         for _, flow := range flows {
27             if !flowInWindow(flow, win) {
28                 continue
29             }
30             stat := perDst[flow.Stats.DstIP]
31             stat.total++
32             switch flow.Stats.ConnState {
33             case "S0":
34                 stat.s0++
35             case "REJ":
36                 stat.rej++
37             }
38             perDst[flow.Stats.DstIP] = stat
39         }
40
41         // Проверяем каждый dstIP на предмет атаки
42         for dst, stat := range perDst {
43             failed := stat.s0 + stat.rej
44             if failed >= d.MinFailedFlowsPerDst &&
45                 float64(failed)/float64(stat.total) >= d.FailedRatio {
```



```

46         alerts = append(alerts, DDoSAlert{
47             Window: win, TargetIP: dst,
48             FlowsTotal: stat.total,
49             FlowsS0: stat.s0, FlowsREJ: stat.rej,
50         })
51     }
52 }
53 }
54 return alerts
55 }

```

Куда вставить — расширить существующий DDoSDetector новым методом, в main.go вызвать его дополнительно к AnalyzeWindows.

5.5 Экспорт в Zeek conn.log формат

Что добавляем — новый файл src/packet/conn_log.go, функция ExportConnLog(filename, flows), выводящая наши потоки в формате, читаемом утилитами zeek-cut, bro-cut, и нативно загружающимся в SIEM-системы.

Формат — TSV с шапкой #fields и фиксированной последовательностью колонок:

```

1 #fields  ts  uid  id.orig_h  id.orig_p  id.resp_h  id.resp_p  proto
2          duration  orig_bytes  resp_bytes  conn_state  history
3          orig_pkts  resp_pkts

```

Все поля у нас уже есть, кроме uid (можно использовать CommunityID или сгенерировать UUID) и history (опционально, выводимо из последовательности флагов).

Ценность — без какой-либо ML или AI наш инструмент сразу становится совместим со всей экосистемой инструментов сетевой безопасности. Это не «красиво для отчёта», это реальная многократная окупаемость одного дня работы.

6 Выводы и рекомендуемые приоритеты

Главное архитектурное наблюдение

Из разобранных решений по уровню совместимости с нашим проектом ситуация однозначная: Zeek подходит, Suricata нет. Zeek работает на flow-уровне, и его идеи переносятся почти напрямую. Suricata построена вокруг сигнатурного DPI, и без расширения нашего парсера до payload-доступа большая часть её ценности нам недоступна.

Это значит, что в ближайшем roadmap'e приоритет — на портировании Zeek-подходов. Сигнатурный путь (Suricata, YARA, JA3/JA4) стоит оставить как отдельную фазу проекта — после того как будет принято решение о расширении парсера до DPI.

Что нужно сделать в текущем спринте

В порядке приоритета, по соотношению ценность/трудозатраты:

1. Connection state code в FlowStats — 1–2 дня. Сразу делает все остальные детекторы выразительнее. Без этого пункта остальные не получится сделать качественно.
2. Community ID в FlowInfo — половина дня. Открывает интероп с экосистемой.
3. Port Scan Detector через TRW — 2–3 дня. Закрывает один из заявленных, но не реализованных детекторов.
4. DDoS Detector по S0/REJ — 1–2 дня. Расширяет существующий детектор реальным механизмом отлова SYN flood и распределённого скана.

5. Экспорт в Zeek conn.log формате — 1 день. Бесплатно открывает интероп.

Итого 6–9 рабочих дней на серьёзное усиление детекторного слоя без ломающих архитектурных изменений.

Что отложить в отдельный roadmap

DPI-направление (декодирование DNS/HTTP/TLS, JA3/JA4, YARA-сигнатуры) — большая работа, требует отдельного решения о расширении парсера. Это самостоятельная веха проекта, и её стоит планировать отдельно.

Что не имеет смысла внедрять

Inline-блокирование через iptables или nftables — несовместимо с оффлайн-анализом записанных pcap. Полная замена нашего движка чужим (Suricata или Zeek как процесс с импортом результатов) — теряется смысл собственного движка.

Источники

- Suricata — <https://suricata.io>, документация, ET Open ruleset (<https://rules.emergingthreats.net/open/>).
- Zeek — <https://zeek.org>, политики `scan.zeek` и `protocols/*`, спецификация Community ID (`corelight/community-id-spec`).
- Jung J., Paxson V., Berger A.W., Balakrishnan H. “Fast Portscan Detection Using Sequential Hypothesis Testing”, IEEE S&P 2004 — алгоритм TRW.
- Zeek conn.log conn_state values — документация <https://docs.zeek.org/en/master/logs/conn.html>.